MAISON BLANC

Deployment Document

Laravel 11 · Vue 3 Admin SPA · MySQL

Document	Deployment & Hosting Guide
Project	laravel-vue-ecommerce (clothing e-commerce)
Version	1.0
Date	15 October 2026
Scope	Architecture, structure, flow, hosting options, interlinking, step-by-step deployment

PATTERN	BACKEND	FRONTEND	DATABASE	AUTH
Modular monolith + SPA	PHP 8.2 / Laravel 11	Blade + Vue 3	MySQL 8	Breeze + Sanctum

Contents

1	Executive Summary
2	Architecture Overview — Monolith or Microservices?
3	Project Structure (Frontend & Backend)
4	Request & Data Flow
5	Deployment Topology — One Platform or Many?
6	Platform Recommendations (Free & Paid)
7	Step-by-Step Deployment Walkthrough
8	Environment Variables Matrix
9	Post-Deployment Verification Checklist
A	Appendix A — Route Map
B	Appendix B — Data Model

1. Executive Summary

This document describes how to deploy the **laravel-vue-ecommerce** application, its structure, its runtime flow, the platforms it can be hosted on, and exactly how the pieces are interlinked when they live on **different platforms**.

Key conclusions

- **Architecture:** a **modular monolith** (one Laravel application serving both the server-rendered storefront *and* the REST API) with a **decoupled Vue 3 admin SPA**. It is **not** microservices.
- **Deployable units:** only **two** code units — (1) the Laravel app (storefront + API) and (2) the Vue admin SPA. They share **one database** and **one file store**.
- **Simplest hosting:** put **everything on one origin** (Laravel app + admin SPA served from the same domain). This eliminates CORS and cookie/domain complexity.
- **Split hosting is possible:** host the API on a PHP/Docker host and the admin SPA on a static host (Netlify/Cloudflare/Vercel). This requires configuring `VITE_API_BASE_URL`, `config/cors.php`, and `APP_URL` — all covered in §7.6.
- **Interlinking mechanism:** the storefront is server-side (same origin as the API by design); the SPA connects over HTTP using a **base URL env variable** and a **Sanctum Bearer token**.

> **Recommended path:** single Docker host (Laravel + nginx) with the admin SPA served under `/admin`, backed by a managed MySQL. Add object storage for persistent uploads. Details in §5.1 and §7.

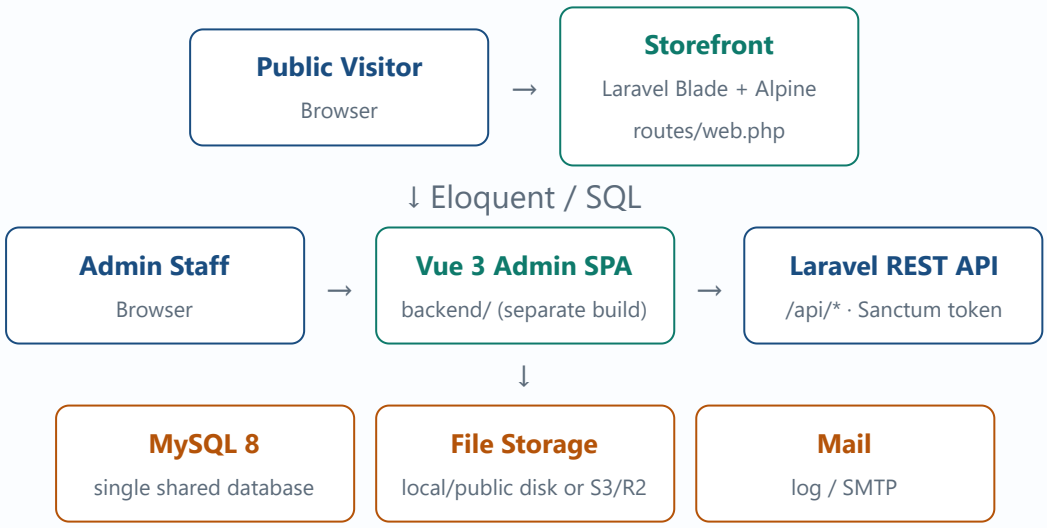
2. Architecture Overview — Monolith or Microservices?

2.1 The verdict

Question	Answer
Is it monolithic?	**Yes — a modular monolith.** All backend concerns (storefront controllers, REST API, models, auth, mail) live in **one** Laravel codebase, deploy, and process.
Is it microservices?	**No.** There are no independently deployable services with separate databases or inter-service messaging.
Is the frontend separate?	**Partly.** The **storefront** is rendered by Laravel (not a separate app). The **admin panel** is a **separate Vue SPA** that consumes the Laravel API.
How many things do you deploy?	**Two units** (Laravel app, Vue SPA) + one database + one file store. The two units can be co-hosted or split.

> **Pattern name:** *Modular monolith with a decoupled admin SPA* — sometimes called a "Laravel + headless-ish admin" or "BFF" pattern. It gives you the operational simplicity of a monolith with a modern SPA for back-office users.

2.2 Component diagram



2.3 Runtime units & their boundaries

Unit	Technology	Talks to	Deployable independently?
Laravel app (storefront + API)	PHP 8.2 / nginx / php-fpm	MySQL, file store, mail	Yes
Admin SPA	Vue 3 static bundle	Laravel API over HTTPS	Yes
Database	MySQL 8 (or SQLite)	—	Yes (managed)
File store	Local disk or S3/R2	—	Yes (managed)

3. Project Structure (Frontend & Backend)

The repository contains **two frontends** (storefront + admin) and **one backend** (Laravel) in a single tree.

```
laravel-vue-ecommerce/
├── app/                        # BACKEND (Laravel)
│   ├── Enums/                  # OrderStatus, PaymentStatus, AddressType, CustomerStatus
│   ├── Helpers/Cart.php        # guest cookie cart + authenticated DB cart
│   ├── Http/
│   │   ├── Controllers/        # Storefront controllers (Home, Cart, Checkout, Order, Profile)
│   │   ├── Controllers/Api/    # Admin REST API controllers (Products, Orders, Settings, ...)
│   │   ├── Middleware/         # Admin.php, GuestOrVerified.php
│   │   ├── Requests/           # validation
│   │   └── Resources/          # API JSON resources
│   ├── Models/                 # Eloquent models
│   └── Support/                # Money.php, Settings.php
├── backend/                    # FRONTEND #2 — Vue 3 admin SPA (own Vite app)
│   ├── src/{components,views,store,router,axios.js}
│   ├── .env.example            # VITE_API_BASE_URL
│   └── vite.config.js
├── resources/
│   ├── css/app.css, js/app.js  # FRONTEND #1 — storefront Vite inputs
│   └── views/                  # Blade templates
├── routes/{web.php, api.php, auth.php}
├── database/{migrations, seeders, factories}
├── public/                     # web root: index.php, build/, storage symlink
├── config/                     # Laravel config
├── tests/                      # PHPUnit (Unit + Feature)
├── Dockerfile                  # php-fpm image
├── docker-compose.yml          # local dev stack
├── vite.config.js              # storefront Vite config
└── composer.json / package.json
```

Backend (Laravel)

- Serves **HTML** (storefront) and **JSON** (API) from one app.
- Entry point: `public/index.php`.
- Routes: `web.php` (sessions) & `api.php` (tokens).
- Middleware config in `bootstrap/app.php` (Laravel 11).

Frontend (two builds)

- **Storefront:** Blade + Alpine + Tailwind → `public/build`.
- **Admin SPA:** Vue 3 + Vuex + Router → `backend/dist`.
- Each has its own `package.json` and Vite config.

4. Request & Data Flow

4.1 Storefront (server-rendered, same origin)

```
Browser ──GET / ──► Laravel router (web.php)
                      └─► ProductController@index
                            ├─ Eloquent: products, banners, categories
                            └─► Blade view + Vite assets (public/build)
Browser ◄── HTML ─────┘

Add to cart ─► Cart helper ─► cookie (guest) OR cart_items table (auth user)
Checkout    ─► validate method ─► create Order + OrderItems + Payment ─► confirmation
Payment     ─► upload proof ─► stored on public disk ─► admin verifies
```

4.2 Admin SPA (decoupled, token-based)

```
Vue SPA ──POST /api/login {email,password}──► AuthController
                                                └─► { user, token }  (Sanctum)
Vue SPA stores token in sessionStorage
Vue SPA ──GET /api/products  (Authorization: Bearer <token>)──► ProductController
                                                └─► JSON (Resource collection)
On 401 ─► axios interceptor clears token ─► redirect to login
```

> **Auth split:** storefront users use the **web session guard** (Breeze); admin users use **API tokens** (Sanctum) and must have `is_admin = 1` (enforced by the `admin` middleware).

4.3 Order & payment lifecycle

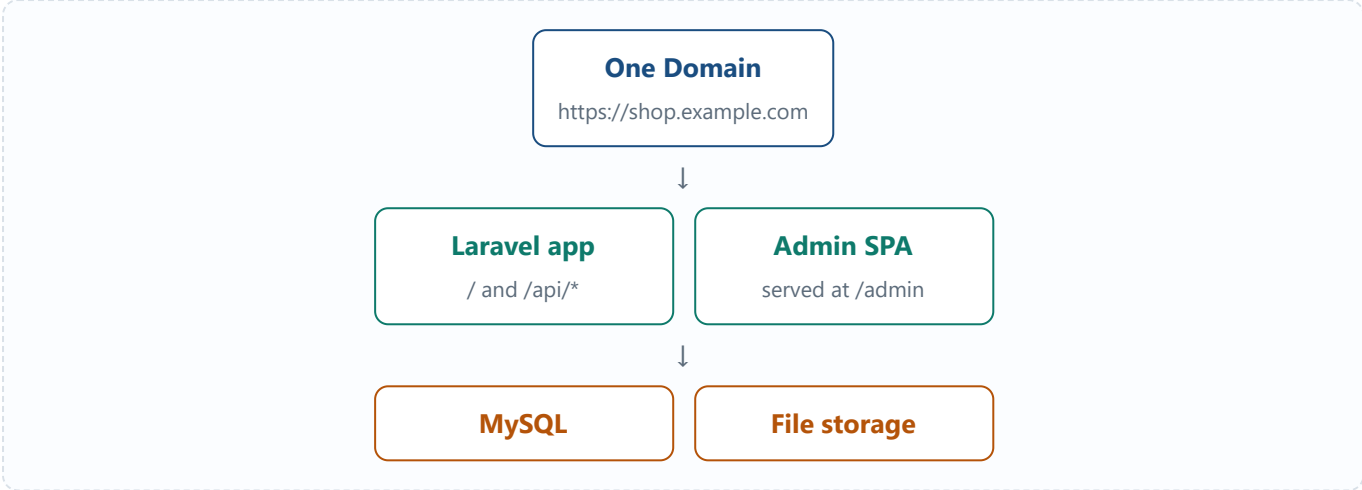
```
Order:   processing ──► paid ──► shipped ──► completed
              │
              └──► cancelled (stock restored)

Payment: pending ──► paid  |  failed
```

5. Deployment Topology — One Platform or Many?

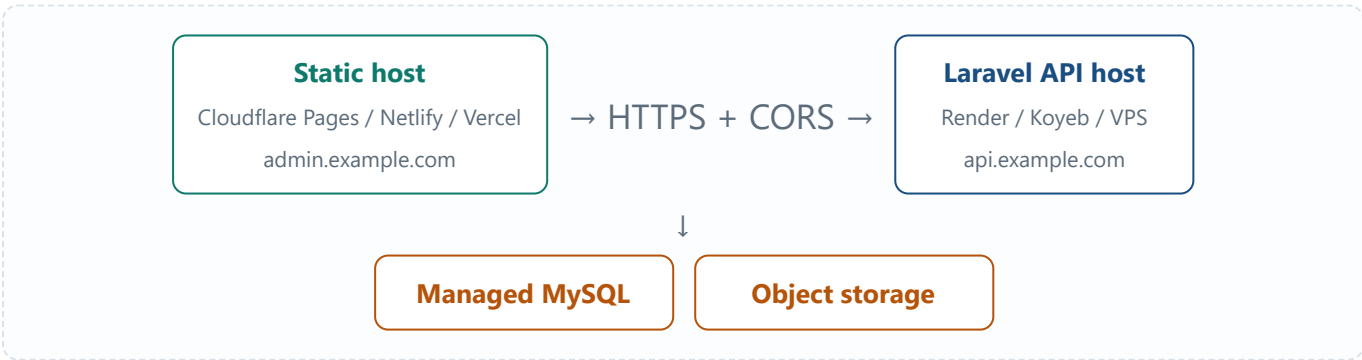
Because the app is a monolith + SPA, you have three practical topologies. Choose based on how much operational simplicity vs. separation you want.

5.1 Option A — Single origin (all-in-one) RECOMMENDED



- **Benefits:** no CORS; cookies/sessions just work; one URL; one SSL cert; simplest ops.
- **Requirements:** the build step copies `backend/dist` into `public/admin`; set the SPA base path to `/admin` (see §7.6.1).
- **Best for:** almost all real deployments, including free tiers.

5.2 Option B — Split deployment (API host + static SPA host)



- **Benefits:** SPA on a fast global CDN; independent scaling; matches frontend/backend team split.
- **Requirements:** create `config/cors.php`, whitelist the SPA origin, set `VITE_API_BASE_URL` to the API domain, run with HTTPS — see §7.6.2.
- **Best for:** when you explicitly want the admin served from a CDN or a separate team owns it.

5.3 Option C — Container platform (single image)

Build one Docker image containing nginx + php-fpm + the compiled storefront *and* admin SPA, then run it on any container host. Internally identical to Option A, but portable/portable across providers. Steps in §7.2.

5.4 Comparison matrix

Criterion	A — Single origin	B — Split	C — Single container
Setup complexity	Low	Medium	Medium

Needs CORS	No	**Yes**	No
Independent SPA scaling	No	Yes	No
CDN for admin	Optional	Yes	Optional
Free-tier friendly	**Yes**	Yes	Yes
Recommended	✓	Advanced	Portable

6. Platform Recommendations (Free & Paid)

6.1 Application host (Laravel)

Platform	Free tier	Notes
Render (Docker)	Yes (sleeps when idle)	Free SSL, easy Docker, cold starts. Recommended
Koyeb (Docker)	Yes (1 nano)	Scale-to-zero, Docker-first.
Railway	Trial credit	Persistent volumes; not free forever.
Fly.io	Usage-based	Global, needs payment method.
VPS (Hetzner/DigitalOcean)	No (~€4–6/mo)	Full control; use the Docker image.
Shared PHP hosting	Yes	No SSH/Node; upload prebuilt files only.

6.2 Database

Option	Free	Notes
Aiven MySQL	Yes (1 GB)	Requires TLS; Recommended
TiDB Cloud Serverless	Yes (5 GB)	MySQL-compatible; TLS.
SQLite	Yes	Zero setup; ephemeral on free hosts.
PlanetScale / RDS	No	Production-grade managed MySQL.

6.3 Admin SPA host (for Option B)

Platform	Free	Notes
Cloudflare Pages	Yes	Global CDN, free SSL. Recommended
Netlify	Yes	Simple SPA deploys.
Vercel	Yes	Fast; good DX.

6.4 File storage

Option	Free	Notes
Cloudflare R2	Yes (10 GB, no egress)	S3-compatible; Recommended
Local `public` disk	—	Ephemeral on free hosts; fine for demos.
AWS S3 / DO Spaces	No	Production standard.

7. Step-by-Step Deployment Walkthrough

7.1 Prerequisites

- Source in a remote Git repository (GitHub).
- A host account (Render/Koyeb) and, optionally, a managed MySQL account (Aiven).
- Local tools to build assets: Node 20 + npm, and a valid `.env.example` in place.
- Optional: an S3/R2 bucket for uploads and an SMTP provider for order emails.

7.2 Build the Laravel application image

Provide a production Dockerfile with three stages: build storefront + admin assets, install Composer deps, then a runtime image with nginx + php-fpm.

```
# Stage 1 — assets
FROM node:20-alpine AS assets
WORKDIR /app
COPY package*.json ./ ; RUN npm ci
COPY resources ./resources ; COPY vite.config.js tailwind.config.js postcss.config.js ./
RUN npm run build
WORKDIR /app/backend
COPY backend/package*.json ./ ; RUN npm ci
COPY backend/ ./ ; RUN npm run build     # -> backend/dist

# Stage 2 — composer
FROM composer:2 AS vendor
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-interaction --no-scripts --prefer-dist --optimize-autoloader

# Stage 3 — runtime
FROM php:8.2-fpm
RUN apt-get update && apt-get install -y --no-install-recommends \
      nginx supervisor unzip libpng-dev libonig-dev libzip-dev libxml2-dev \
    && docker-php-ext-install pdo_mysql mbstring zip bcmath gd fileinfo \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /var/www
COPY . .
COPY --from=vendor /app/vendor ./vendor
COPY --from=assets /app/public/build ./public/build
COPY --from=assets /app/backend/dist ./public/admin     # admin SPA at /admin
COPY docker/production/nginx.conf /etc/nginx/sites-available/default
COPY docker/production/supervisord.conf /etc/supervisor/conf.d/app.conf
COPY docker/production/entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh \
    && mkdir -p storage/framework/{cache,sessions,views} bootstrap/cache storage/logs \
    && chown -R www-data:www-data storage bootstrap/cache
EXPOSE 8080
ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
```

Entrypoint (runs on every boot)

```
#!/bin/sh
set -e
php artisan storage:link || true
```

```
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
exec supervisord -c /etc/supervisor/conf.d/app.conf   # starts php-fpm + nginx
```

7.3 Deploy the backend

1. Push the repo to GitHub.
2. **Render** → **New Web Service** → connect the repo → **Docker** → Dockerfile `Dockerfile.production` → plan **Free**.
3. Add environment variables (§8), especially `APP_KEY`, `APP_URL`, and DB credentials.
4. Deploy. The entrypoint runs migrations automatically.
5. Seed initial data once (Render Shell / one-off job):

```
php artisan db:seed --force
php artisan db:seed --class=ClothingStoreSeeder --force
php artisan db:seed --class=BannerAndSettingsSeeder --force
```

7.4 Database setup

- **Managed MySQL (recommended):** create the DB in Aiven, copy host/port/user/pass/db into env vars. Aiven requires TLS — add the CA option in `config/database.php`:

```
'options' => extension_loaded('pdo_mysql') ? array_filter([
    PDO::MYSQL_ATTR_SSL_CA => env('MYSQL_ATTR_SSL_CA'),
]) : [],
```

- **SQLite (fastest demo):** `DB_CONNECTION=sqlite`, `DB_DATABASE=/var/www/database/database.sqlite`; `touch` the file in the entrypoint. Data resets on redeploy on ephemeral hosts.

7.5 File storage setup

Uploads (product images, banners, logos, payment proofs) go through Laravel's filesystem disk.

- Default: local `public` disk → run `php artisan storage:link` at boot.
- Persistent: configure the `s3` disk for Cloudflare R2 and set `FILESYSTEM_DISK=s3`.

7.6 Interlinking the frontend and backend

This is the part that decides whether your split works. There are two scenarios.

7.6.1 Same origin (Option A) — the admin SPA under `/admin`

Change three things so the SPA is served from the Laravel domain at `/admin`:

backend/vite.config.js

```
export default defineConfig({
  plugins: [vue()],
  base: '/admin/',          // served under /admin
  server: { /* dev proxy unchanged */ },
})
```

backend/src/router/index.js

```
const router = createRouter({
  history: createWebHistory('/admin/'),   // base path
  routes
})
```

backend/.env.production

```
VITE_API_BASE_URL=https://shop.example.com
```

The Docker build copies `backend/dist` → `public/admin`, so the SPA and API share the origin. **No CORS needed.**

7.6.2 Split origin (Option B) — SPA on a CDN, API elsewhere

a) Point the SPA at the API (`backend/.env.production`):

```
VITE_API_BASE_URL=https://api.example.com
```

Keep the router at root (`createWebHistory()`, no base) since it is served at the CDN root.

b) Allow the SPA origin in CORS — create `config/cors.php`:

```
<?php
return [
    'paths' => ['api/*', 'sanctum/csrf-cookie'],
    'allowed_methods' => ['*'],
    'allowed_origins' => [env('ADMIN_URL', 'https://admin.example.com')],
    'allowed_headers' => ['*'],
    'supports_credentials' => true,
];
```

Then set `ADMIN_URL=https://admin.example.com` on the API host.

c) Trust the platform proxy & force HTTPS URLs — in `bootstrap/app.php`:

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->trustProxies(at: '*');
})
```

d) Build & deploy the SPA as a static site (Cloudflare Pages/Netlify/Vercel):

```
cd backend
npm ci
npm run build          # output in backend/dist
# publish backend/dist (SPA redirect all fallback to index.html)
```

7.6.3 How the pieces find each other (summary)

Link	Mechanism	Where set
Storefront → Laravel	Same origin (SSR)	nginx/web root
SPA → API	`VITE_API_BASE_URL` baked at build	`backend/.env.production`
API trusts SPA origin (split only)	`config/cors.php` + `ADMIN_URL`	Laravel config/env
SPA auth	Sanctum Bearer token	`sessionStorage` in SPA
Public URLs / HTTPS	`APP_URL` + `trustProxies`	env + `bootstrap/app.php`
Media served	`storage:link` or S3/R2 URL	entrypoint / env

8. Environment Variables Matrix

8.1 Laravel backend (`.env`)

Variable	Example	Purpose
`APP_ENV`	production	Disables debug in prod
`APP_DEBUG`	false	Never show stack traces publicly
`APP_KEY`	base64:...	Encryption / sessions
`APP_URL`	https://shop.example.com	URL generation, links
`DB_CONNECTION`	mysql / sqlite	Database driver
`DB_HOST/PORT/DATABASE/USERNAME/PASSWORD`	...	DB credentials
`MYSQL_ATTR_SSL_CA`	/etc/ssl/certs/...	Managed MySQL TLS
`FILESYSTEM_DISK`	local / s3	Upload storage
`AWS_*`	bucket/keys/endpoint	R2/S3 storage
`ADMIN_URL`	https://admin.example.com	CORS (split only)
`LOG_CHANNEL`	stderr	Platform-friendly logs
`MAIL_*`	smtp creds	Order emails

8.2 Admin SPA (`backend/.env`)

Variable	Example	Purpose
`VITE_API_BASE_URL`	https://api.example.com	API base for axios

> **Never commit `.env` files.** Only `.env.example` belongs in Git. After editing env vars on a running host, clear caches: `php artisan config:clear && php artisan config:cache` .

9. Post-Deployment Verification Checklist

✓	Check
☐	`APP_KEY` set; `APP_DEBUG=false`; `APP_ENV=production`
☐	HTTPS active; `APP_URL` uses `https`; `trustProxies` enabled
☐	Migrations ran; catalog + admin seeded
☐	`storage:link` ran; uploads display
☐	Default admin password changed
☐	Storefront: browse → cart → register/verify → checkout → place order
☐	Pay page loads; proof upload stored; admin can view proof
☐	Admin SPA: login → each section loads (Products, Orders, Settings, Payment Methods)
☐	(Split) CORS returns the SPA origin; preflight `OPTIONS` OK
☐	(Split) `VITE_API_BASE_URL` matches the deployed API domain
☐	Upload persistence strategy chosen (local ephemeral vs R2/S3)
☐	SMTP configured if order emails are required

Appendix A — Route Map

A.1 Storefront (web)

Method	URI	Name	Purpose
GET	/	home	Home (banners, categories, products)
GET	/category/{slug}	byCategory	Category listing
GET	/product/{slug}	product.view	Product detail
GET/POST	/cart, /cart/add, /cart/remove, /cart/update-quantity	cart.*	Cart operations
GET/POST	/profile, /profile/password-update	profile*	Profile & addresses
GET/POST	/checkout, /checkout/place, /checkout/placed/{order}	checkout*	Checkout & order placement
GET/POST	/orders, /orders/{order}, /orders/{order}/pay	order.*	My orders & payment proof
—	/register, /login, /forgot-password, /verify-email, …	—	Breeze auth

A.2 Admin API (`/api`, Sanctum + admin)

Method	URI	Purpose
POST	/api/login	Admin login → token
GET	/api/user	Current admin
POST	/api/logout	Revoke token
CRUD	/api/products · /api/users · /api/customers · /api/categories (+/tree) · /api/banners · /api/payment-methods	Admin management
GET/PUT	/api/settings	Business settings
GET	/api/countries	Country list
GET/POST	/api/orders, /api/orders/{order}, change-status, change-payment-status	Orders & statuses
GET	/api/dashboard/* · /api/report/*	Dashboard & reports

Appendix B — Data Model

```
users (id, name, email, password, is_admin, email_verified_at)
  └─1:1─ customers (PK user_id, first_name, last_name, phone, status)
           └─1:N─ customer_addresses (type, address..., country_code → countries.code)

users 1:N orders
orders (id, total_price, status, created_by, updated_by)
  ├─1:N─ order_items (order_id, product_id, quantity, unit_price)
  └─1:1─ payments (order_id, amount, status, type, session_id, payment_proof)

categories (id, name, slug, active, parent_id[self], image)
  └─N:M─ products  (pivot: product_categories)

products (id, title, slug, description, price, quantity, published)
  └─1:N─ product_images (product_id, path, url, mime, size, position)

cart_items (user_id, product_id, quantity)
banners (title, kicker, text, image_url, link_label, link_url, position, sort_order, active)
settings (key, value)
payment_methods (name, code, description, details, is_active, sort_order)
countries (code PK, name, states json)
personal_access_tokens (Sanctum)
```

Enums

- **OrderStatus:** unpaid, processing, paid, cancelled, shipped, completed
- **PaymentStatus:** pending, paid, failed
- **AddressType:** shipping, billing
- **CustomerStatus:** active, disabled

> **End of document.** For local development, storefront flow, seeder details and testing, see the companion *Website Documentation* and *Free Hosting & Launch Guide*.